

Übungsaufgaben – Vergleichsfunktionen (IHK-Prüfungsstil)

Hinweis: Nur die Aufgabe „Winter 2023/24“ entspricht direkt einer echten IHK-Aufgabe (verkürzt formuliert). Die anderen Aufgaben sind inhaltlich typische, aber als Übung formulierte Aufgaben, angelehnt an frühere Jahrgänge.

1. Vergleich von Mitarbeitern nach Gehalt – Winter 2023/24

Jahr: Winter 2023/24 (Fachinformatiker AE – verkürzte Fassung)

Zur Flexibilisierung des Sortieralgorithmus wird eine Funktion benötigt, die ...

- den Wert 0 zurückgibt, wenn die zwei der Funktion übergebenen Objekte gleich groß sind,
- einen positiven Wert zurückgibt, wenn der erste Parameter größer als der zweite ist und
- einen negativen Wert im verbleibenden Fall.

Erstellen Sie eine Funktion `vergleicheMitarbeiter(Mitarbeiter m1, Mitarbeiter m2)`, die zwei Mitarbeiter-Objekte als Parameter bekommt und einen entsprechenden Rückgabewert liefert. Der Vergleich soll über das Gehalt der Mitarbeiter erfolgen. Die Klasse `Mitarbeiter` stellt dafür die Methode `getGehalt(): Double` zur Verfügung.

Lösung (Pseudocode):

```
funktion vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter) Integer
    diff: Double
    diff ← m1.getGehalt() - m2.getGehalt()

    WENN diff > 0 DANN
        RETURN 1          // m1 verdient mehr
    WENN diff = 0 DANN
        RETURN 0          // gleiches Gehalt
    SONST
        RETURN -1         // m1 verdient weniger
    ENDE WENN
endfunktion
```

این تابع دو شیء `Mitarbeiter` را بر اساس مقدار `getGehalt()` مقایسه می‌کند.
اگر حقوق `m1` بیشتر باشد، مقدار 1 برمی‌گرداند؛ اگر مساوی باشد، مقدار 0؛ و در غیر این صورت مقدار -1.

2. Vergleich von Personen nach Alter – Beispiel (ähnlich älteren Prüfungen, ca. 2019)

Jahr: Übungsaufgabe, angelehnt an ca. Sommer 2019

Schreiben Sie eine Funktion `vergleichePerson(Person p1, Person p2)`, die zwei Personen anhand ihres Alters vergleicht.

Die Methode `getAlter(): Integer` liefert das Alter einer Person.

Die Funktion soll

- 0 zurückgeben, wenn beide Personen gleich alt sind,
- einen positiven Wert zurückgeben, wenn die erste Person älter ist als die zweite,
- einen negativen Wert im übrigen Fall.

Lösung (Pseudocode):

```
funktion vergleichePerson(p1: Person, p2: Person) Integer
    alter1 ← p1.getAlter()
    alter2 ← p2.getAlter()

    WENN alter1 = alter2 DANN
        RETURN 0
    SONST WENN alter1 > alter2 DANN
        RETURN 1
    SONST
        RETURN -1
    ENDE WENN
endfunktion
```

در این تمرین، دو شیء `Person` با استفاده از `getAlter()` (سن) مقایسه می‌شوند و خروجی دقیقاً مانند سؤال کارمند 1، 0 یا -1 است.

3. Vergleich von Schülern nach Note – Beispiel (ähnlich Winter 2020/21)

Jahr: Übungsaufgabe, angelehnt an Winter 2020/21

Entwickeln Sie eine Funktion `vergleicheSchueler(Schueler s1, Schueler s2)`. Der Vergleich soll über die Zeugnisnote erfolgen. Die Methode `getNote(): Double` liefert die Note (1.0 = sehr gut, 6.0 = ungenügend).

Die Funktion soll:

- 0 zurückgeben, wenn beide Schüler die gleiche Note haben,
- einen positiven Wert zurückgeben, wenn der erste Schüler eine schlechtere Note hat (größere Zahl),
- einen negativen Wert, wenn der erste Schüler eine bessere Note hat.

Lösung (Pseudocode):

```
funktion vergleicheSchueler(s1: Schueler, s2: Schueler) Integer
    note1 ← s1.getNote()
    note2 ← s2.getNote()

    diff ← note1 - note2

    WENN diff = 0 DANN
        RETURN 0
    SONST WENN diff > 0 DANN
        // s1 hat die schlechtere (größere) Note
        RETURN 1
    SONST
        // s1 hat die bessere (kleinere) Note
        RETURN -1
    ENDE WENN
endfunktion
```

در اینجا نمره‌ها هرچه کوچک‌تر باشند بهترند. بنابراین اگر اختلاف مثبت باشد یعنی s1 نمره بدتری دارد و مقدار 1 برمی‌گردد؛ اگر منفی باشد، s1 نمره بهتری دارد و -1 برمی‌گردد.

4. Generische Vergleichsfunktion – Beispiel zu Generics (ähnlich Sommer 2022)

Jahr: Übungsaufgabe, angelehnt an Sommer 2022

Zur Erstellung eines flexiblen Sortieralgorithmus soll eine generische Vergleichsfunktion eingeführt werden. Erstellen Sie eine Funktion `vergleiche(a: T, b: T)`, die für beliebige Objekte vom Typ `T` einen Integer-Wert zurückgibt. Es soll angenommen werden, dass es für den Typ `T` eine Methode `compareTo(b: T): Integer` gibt, die

- 0 liefert, wenn beide Objekte gleich sind,
- einen positiven Wert, wenn `a` größer als `b` ist,
- einen negativen Wert sonst.

Lösung (Pseudocode, generisch):

```
funktion vergleiche(a: T, b: T) Integer
    RETURN a.compareTo(b)
endfunktion
```

این تمرین نشان می‌دهد که چگونه می‌توان با استفاده از نوع جنریک `T` یک تابع مقایسه‌گر عمومی نوشت که برای هر کلاسی که `compareTo` را پیاده‌سازی کرده است قابل استفاده باشد.